

TKG Test ROM Manual

Last revised: 06 September 2026

Written by: SNESNESCUBE64

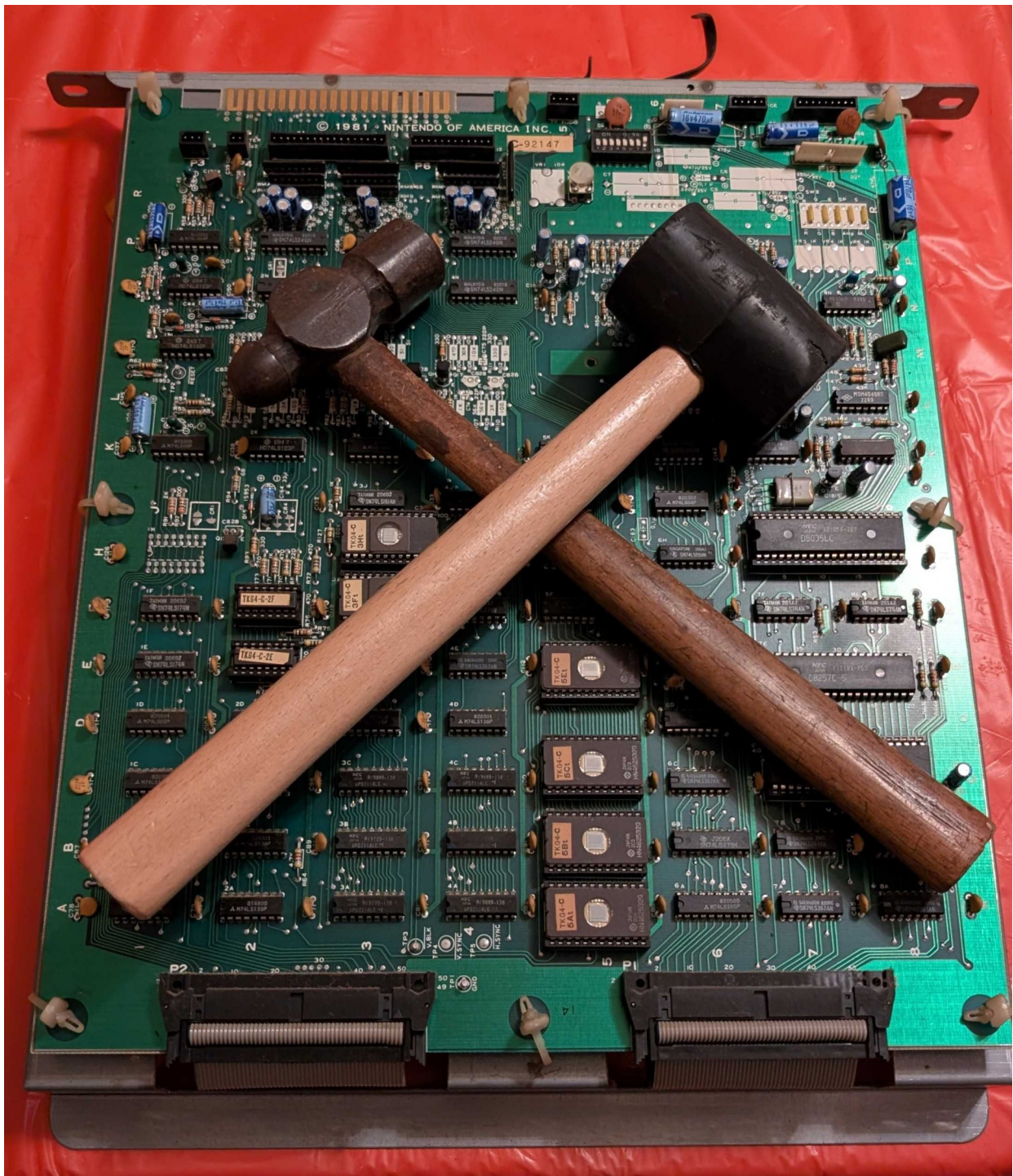


Table of Contents

Chapter 1: Overview.....	3
Requirements.....	3
Compatible Hardware.....	3
Using the Test ROM.....	3
Chapter 2: Startup Tests.....	5
RAM Tests.....	5
RAM IC Tests.....	5
RAM Bank Test.....	5
Brief Explanation of RAM Functionality.....	5
RAM Failure Tones.....	6
TRS2 and TKG2 RAM Locations.....	6
TKG3 RAM Locations.....	7
TKG4 RAM Locations.....	7
ROM Checksums.....	7
Test ROM Integrity Test.....	7
ROM Locations.....	8
ROM Checksum Table.....	8
DMA Test.....	8
NMI Test.....	9
Audio Tests.....	9
Transition to Runtime.....	9
Chapter 3: Runtime Tests.....	10
Controls Test.....	10
DIP Switch Test.....	10
Palette Test.....	11
Screen Invert.....	11
Monitor Test.....	11
Pattern 0 – Checker Board.....	12
Pattern 1 – Grid.....	12
Pattern 2 – Block.....	12
Sound Test.....	13
Music Test.....	13
Background Test.....	14
Sprite Test.....	14
Reset.....	14
Chapter 4: Test Architectures.....	15
Appendix A: Abbreviations.....	16
Appendix B: ICs mentioned.....	17

Chapter 1: Overview

Requirements

This test ROM will not work under every condition. Some things need to be taken into consideration:

- All boards involved must have power, in this case +5V, +12V, and -5V must all be present and within appropriate tolerance, usually within +/- 5%.
- Clocks must be present enough to run. The CPU runs at approximately 3MHz.
- The reset pin and NMI pins must not be held in active. If both are always active, then the ROM will not boot.
- The CPU must be able to access address 0x0000 to 0x0FFF. If the game does not boot it cannot access these ROMs.
- Address and data buffers must be working, if they are not it will not be possible for the program to run.
- Audio OR video must be working. If neither are working, the test ROM will have no way to report failures
- The ROM socket for 5F or 5E (depending on which hardware being tested) must be in good enough condition to make proper contact.

Compatible Hardware

This test ROM is intended for original TKG hardware. It is compatible with the following hardware:

- Fully compatible
 - TKG2
 - TKG3
 - TKG4
- Partially compatible
 - TRS2 – RAM, ROM, DMA, and NMI tests should all work. However, it is a different game so audio tests will not work. Monitor tests might also not work correctly.

This test ROM will work in the cocktail, cabaret, and upright models of the machine.

Using the Test ROM

Using the test ROM is very simple. The ROM is burnt onto a 2532 4k EPROM. In order to run the tests you place the EPROM into location 5F for TKG2/TKG3 hardware and 5E TKG4 hardware. Note the notch in the socket, EPROM, and pin 1 markers. After the ROM is inserted reconnect the board back to the cabinet or bench setup for testing.

There are two groups of tests performed by the test ROM: Startup and Runtime. Startup tests are tests that are automatically run when the game boots. This tests RAM, ROM, DMA, NMI, and Audio tests. All tests with the exception of audio tests must pass in order to transition from startup to runtime. Runtime tests are additional tests used for diagnosing other issues with the machine in question. It contains things such as a monitor test and control printouts.

TKG START-UP TESTS V0.15 22 AUG 2026				TKG RUN-TIME TESTS V0.15 22 AUG 2026			
RAM TESTS				CONTROLS TEST			
RAM 0L	PASS	RAM 3L	PASS	P1 RIGHT	OFF	P2 RIGHT	OFF
RAM 0H	PASS	RAM 3H	PASS	P1 LEFT	OFF	P2 LEFT	OFF
RAM 1L	PASS	RAM 4L	PASS	P1 UP	OFF	P2 UP	OFF
RAM 1H	PASS	RAM 4H	PASS	P1 DOWN	OFF	P2 DOWN	OFF
RAM 2L	PASS			P1 JUMP	OFF	P2 JUMP	OFF
RAM 2H	PASS	B. TEST	PASS	P1 START	OFF	P2 START	OFF
				COIN	OFF		
ROM CHECKSUMS				DIP SW TEST			
ROM 0	F31E			00000001			
ROM 1	73BA			ABCDEFGH			
ROM 2	B2BC						
ROM 3	A0F0						
SYSTEM TESTS				SELECT A TEST			
DMA	PASS			? PALETTE TEST			1
NMI	PASS			SCREEN INVERT			0
				MONITOR TEST			0
				SOUND			0
				MUSIC			0
				BACKGROUND TEST			
				SPRITE TEST			
				RESET			
AUDIO TESTS							
SOUND	6						
MUSIC	4						

Limitations

The test ROM is unable to directly test every component on the board but rather is meant to test functionality and sections. The board has to be able to run code in order for it to execute at all which means the power on reset circuit, clocks, and ROM addressing circuitry must be working. If it cannot read any data, verify that it can access the ROMs and data is not being corrupted on the way back.

This manual also mentions many ICs, it will not mention every part of a circuit for every combination of TKG hardware. It will do its best to explain how it works but reference the schematics for specifics. This test ROM, if it can execute, will do its best to give clues on what is going wrong. Note that this will not be explaining the clock, ECL, or video/sprite circuitry that much farther than the RAM covered by the test. That is still up to the technician to figure out if there is an issue as it falls outside the scope of the test ROM.

Chapter 2: Startup Tests

RAM Tests

There are two different sections of RAM tests: RAM test and RAM bank test. The RAM test tests the individual RAM chips themselves. The RAM bank test (labeled B. Test) which tests the addressing of the RAM test. There are five total banks of RAM ICs. Each bank consists of a pair of two 4-bit RAM chips designated high and low where high is the more significant nibble and low is the less significant nibble. The RAM tests are the first tests that run on the system as they need to be run without using RAM for itself.

Please note: When the RAM test is running you may say “garbage” on the screen. The reason for this is that video and sprite RAM is tested. Whenever data is written to those areas it will appear on the screen automatically. There is no video blanking/suppression that can be controlled via software.

RAM IC Tests

There are two RAM tests that tests the ICs themselves. The first test fills all of RAM with a multiple of \$11. It increments the RAM each pass by \$11 until it does \$FF. It works by filling the RAM first and then checking it after. By that time, it has had enough time for the RAM to become flakey in theory and then test each individual bit. If an error is detected, it sets a corresponding bit to the test results register. In the event there is a RAM failure, eleven tones will play. Refere to the “RAM Failure Tones” section to identify which RAM failed. Also refer to sections following it to see to which RAM IC it corresponds to.

RAM Bank Test

The RAM bank test has to be handled differently. It is possible for all of the RAM tests to pass but still have a failure in addressing. One scenario is only the RAM0 bank is able to be selected. It could still report pass as only one bank is tested at a time. The solution for this is to write a different value to each bank and read it back. If the values are overwritten then there is an addressing/bank issue. The bank test will only process results if all RAM tests are reported as “good”. If there is a RAM failure before, it will print a result of “N.A.” as it would be impossible for it to pass this way. In the event that a bank failure is reported, the 11th tone will be a crash and it will report the failure on the screen.

Brief Explanation of RAM Functionality

On TKG4 hardware, the 74LS138 at 2D is what selects the work RAM banks. It also must be selected by the 74LS138 at 4D in order to properly select the bank. In order to also select the RAM, a read or write has to occur, that signal is generated by the CPU which feeds it to through a 74LS367 buffer. The write signal that is consumed by the work RAM is also acquired this way.

Sprite and video RAM is a bit different. In the schematics sprite RAM is labeled “Obj RAM” or “Object RAM”. Video RAM is just labeled as VRAM. The write signals for both of these RAMs are generated by the 74LS138 at 2C. The read signals for these RAMs are generated by the 74LS138 at 2B. Part of the selection is done by the 74LS138 at 4D. The 74LS139 at 2A and 74LS32 at 1A is involved with the DMA side of the sprite RAM, making the “ObjRq” or “Object Request” signal. TKG hardware

is capable of transferring data. This ObjRq is in part responsible for that. These are for the most part all of what is involved with the sprite and video RAM on the CPU side.

The video PCB side adds an extra layer of complexity as the sprite and video RAM data has to be utilized by the rest of the video board. It does this by switching between video counter signals and CPU address lines depending on the situation. Whenever the CPU needs to access sprite or video RAM, the appropriate 74LS157s switch the address lines from the video counters to the address lines. After that switch the appropriate 74LS245 buffer turns on and gives access to the sprite or video RAM data lines. The sprite RAM is very similarly written to as the video RAM. Refer to the schematics for more information on what chips are used to access the RAM. The only difference is that the RAM is 2148 and it has an additional object request signal to handle. Where the RAMs really differ is how their data is used by the rest of the video board. Video RAM data gets fed into ROMs whose data gets merged back to video out more directly. Sprite RAM has to go through the whole ECL circuit setup for it.

Unfortunately the test ROM can only validate that the CPU has access to the sprite and video RAM. It cannot directly test how the video board interprets that data. However, the background and sprite test can be used to troubleshoot the video PCB functionality.

RAM Failure Tones

In the event there is a RAM error, it will play eleven tones: jump sound indicates a pass, crash indicates a failure. This is reliant on the discretely generated sounds jump and crash being functional. If they are not working it may not sound correct.

Tone #	RAM Test
1	RAM0 Low
2	RAM0 High
3	RAM1 Low
4	RAM1 High
5	RAM2 Low
6	RAM2 High
7	RAM3 (Sprite) Low
8	RAM3 (Sprite) High
9	RAM4 (Video) Low
10	RAM4 (Video) High
11	RAM Bank Test

TRS2 and TKG2 RAM Locations

Name	Designation	Type	Start Address	Size	High Bit Location	Low Bit Location
RAM0	Work	2114	0x6000	1k x 4	C-6L	C-6H
RAM1	Work	2114	0x6400	1k x 4	C-6M	C-6J
RAM2	Work	2114	0x6800	1k x 4	C-6N	C-6K
RAM3	Sprite	2148	0x7000	1k x 4	V-5J	V-5K

Name	Designation	Type	Start Address	Size	High Bit Location	Low Bit Location
RAM4	Video/Background	2114	0x7400	1k x 4	V-3K	V-3J

TKG3 RAM Locations

Name	Designation	Type	Start Address	Size	High Bit Location	Low Bit Location
RAM0	Work	2114	0x6000	1k x 4	C-6L	C-6H
RAM1	Work	2114	0x6400	1k x 4	C-6M	C-6J
RAM2	Work	2114	0x6800	1k x 4	C-6N	C-6K
RAM3	Sprite	2148	0x7000	1k x 4	V-2L	V-2M
RAM4	Video/Background	2114	0x7400	1k x 4	V-5L	V-5M

TKG4 RAM Locations

Name	Designation	Type	Start Address	Size	High Bit Location	Low Bit Location
RAM0	Work	2114	0x6000	1k x 4	C-4C	C-3C
RAM1	Work	2114	0x6400	1k x 4	C-4B	C-3B
RAM2	Work	2114	0x6800	1k x 4	C-4A	C-3A
RAM3	Sprite	2148	0x7000	1k x 4	V-6R	V-6P
RAM4	Video/Background	2114	0x7400	1k x 4	V-2R	V-2P

ROM Checksums

The ROM test performs a 16-bit checksum on the contents of all ROMs. This test does not report the integrity of the ROMs nor does it validate that it is correct for a location. That is up to the user to determine. The exception to this is ROM 0, which is where the test ROM is located.

Test ROM Integrity Test

As part of the ROM test, ROM 0 will be validated. ROM 0 is the test ROM itself. Its own checksum is appended to the footer as part of a post-build script. It will compare the measured checksum with what is expected. If there is any corruption of the test ROM or if there is issues with accessing say the upper addresses, it will report a failure by printing “Integrity Fail”. In this case, the test ROM may not be trustworthy and could run into issues during execution.



ROM Locations

ROM ID	ROM Type	ROM Size	Start Address	TKG2/TKG3 Location	TKG4 Location
ROM 0	2532	4k	0x0000	5F	5E
ROM 1	2532	4k	0x1000	5G	5C
ROM 2	2532	4k	0x2000	5H	5B
ROM 3	2532	4k	0x3000	5K	5A

ROM Checksum Table

ROM ID	US Set 1	US Set 1 Hard	US Set 2	Japan Set 1	Japan Set 2	Japan Set 3
ROM 0	A967	AA53	A967	A967	A967	A967
ROM 1	73BA	73BA	73BA	745D	73BB	745D
ROM 2	B2BC	B2BC	B3AE	B3A9	B3AE	B3AE
ROM 3	A0F0	A205	A384	A490	A490	A4AC

DMA Test

TKG hardware relies heavily on DMA to transfer data from work RAM to sprite RAM. During gameplay it is triggered during the NMI routine. The reason for this is because this is when vertical blanking (also known as VBlank) is occurring, meaning that nothing is being drawn on screen. VBlank occurs sixty times per second. On TKG hardware, an 8257 DMA controller is used to perform these operations as well as some supporting circuitry. On TKG4 hardware, that circuitry includes the 74LS373 octal latch at 6B, the 74ls08 quad and-gate at 5F, and the 74LS125 buffer at 6E. The 8257 derives its clock from the 1H signal (~3MHz). Unlike the CPU clock however, it goes through an inverter (74LS04 at 6D). Chip select circuitry is also involved. Refer to the schematics to find out more.

The DMA test works by initiating a transfer from the ranges \$6900-\$6A80 to \$7000-\$7180. This test mimics what would be observed in real game conditions. This test is run three times with different byte values. After the DMA transfer is kicked off, the CPU reads back the data that should have been copied over and compares it to what is in the work RAM mirror. If all bytes do not match, then the test is marked as “fail” The DMA test can only occur if RAM2 and RAM3 are functional. If either of these RAM banks failed its RAM test, the DMA test will not process the results and instead print N.A.

A severely failing 8257 DMA controller can cause the board to be unable to boot in rare cases. The DMA controller has direct access to the databus shared with the CPU. For the purposes of the test ROM, you should be able to remove the DMA controller to be able to perform startup tests. However, it will not proceed past the startup tests due to the DMA failure. Be aware that the DMA controller is not always in an IC socket.

NMI Test

TKG hardware uses a hardware controlled NMI. NMI is derived from the VBlank signal which pulses at approximately 60Hz. It goes through a 74ls74 flip flop (8F on TKG4 hardware) before ultimately toggling the NMI pin on the CPU. The flip flop acts as an NMI enable. To enable NMI, a non-zero value in the lower 4 bits must be written to \$7D84. To disable the interrupt, write zero to \$7D84. Every time NMI triggers it must be re-enabled. In original TKG software, it gets re-enabled towards the end of the NMI routine.

The test ROM validates that the NMI functionality works. NMI is enabled and when it jumps to the NMI routine it sets a bit in the results register. Immediately after enabling NMI, it enters a wait loop. If the NMI routine does not trigger during wait loop it “times out” and from there it will register it as a failure. It is required for the NMI test to pass in order to progress to the runtime tests.

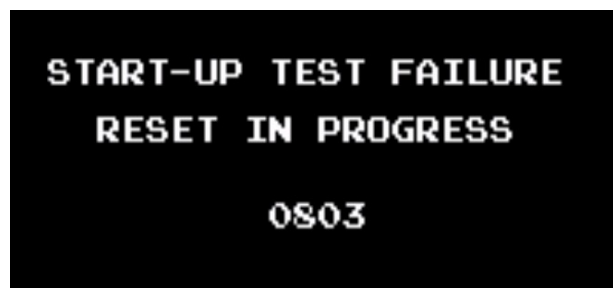
Audio Tests

The audio tests automatically play all available sounds on standard TKG hardware. These do not have any pass/fail criteria. It is up to the operator to determine whether or not it is correct. Refer to the tables in the sound and music sections of the runtime chapter to understand what is supposed to be played. There are 7 sounds (0 to 6) and 15 music tracks (1 to 15).



Transition to Runtime

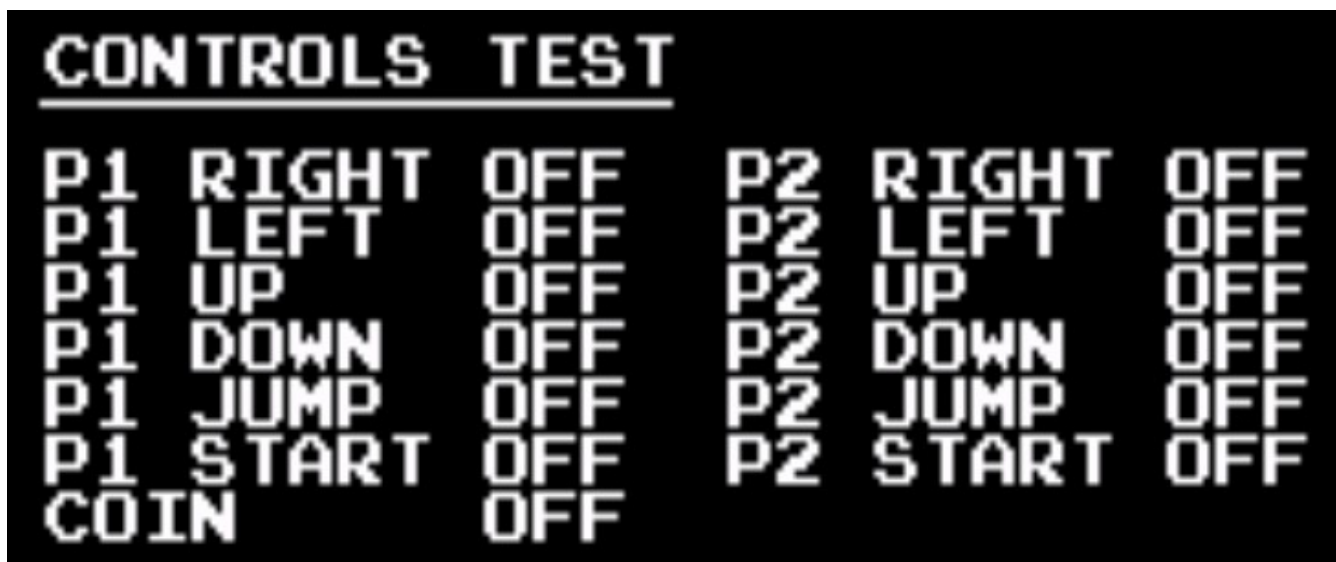
There are scenarios in which the test ROM will not transition to run time. It must pass all RAM tests and the NMI test. If any of these tests do not pass, the test ROM will report a startup test failure and the game will reset. During this time, it will print a message saying that there was a failure and print an error code. The error code translates to the result buffer. Refer to the error decoding sheet to decode this.



Chapter 3: Runtime Tests

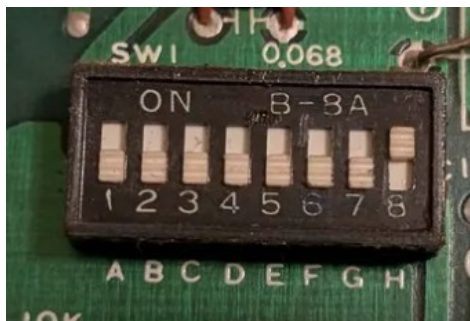
Controls Test

This is a very simple test that just prints the state of the buttons. This does not have its own screen, it just prints the button state as part of the menu. P1 controls are all the controls on the upright and all available controls on player 1 side of the cocktail table. P2 controls are only available on the cocktail table P2 side. The coin switch is a bit unique. TKG hardware has no way to differentiate between the coin switch and the service switch using software. So if either switch is toggled it will register as “coin”. When a button is pushed, it will be represented by value of the button transitioning to “on”. If any of the values are “stuck” then the first place to check would be the pullup resistor arrays and the buffer ICs. There could also be some failures in the addressing circuitry that is enabling the read of the switches.



DIP Switch Test

The DIP switches are a set of eight switches found on the CPU PCB. They are labeled both '1' through '8' and 'A' through 'H'. The test will print the current state of the dip switches. “On ”is represented by a '1'. “Off” is represented by a '0'. To find out what the actual dip switches represent, refer to the game manual. If they are stuck on, like the controls, the first place to check would be the pullup resistors and buffer IC for the dip switches. For more details refer to the schematics on the CPU board.



Palette Test

This is a very simple test that changes the color palette of the background characters. This will be noticeable by the text changing color. The full color palette can be observed using monitor test pattern two. There are four possible color palettes for the background. They can be changed in software by writing to the least significant bit of addresses \$7D86 and \$7D87.

Screen Invert

This is yet another very simple test that just changes the screen inversion status, called flip in the schematics. By default, TKG prints to the screen “upside down”. The actual inversion process is handled through hardware and is enabled by software. To uninvert the screen, write ‘1’ to address \$7D82. To invert the screen, write ‘0’ to \$7D82. On TKG4, it is controlled by the 74LS259 at 5H and goes through an inverter at 5J. From there the flip signal is consumed by several chips on the video board to handle the flipping of individual video sections.



Normal Screen



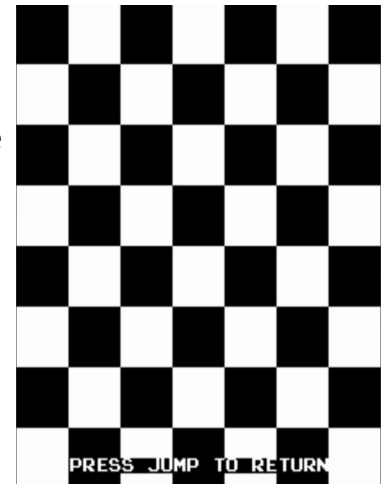
Inverted Screen

Monitor Test

This section will give a brief overview of what patterns are available. For a better understanding do additional research about dialing in the monitor.

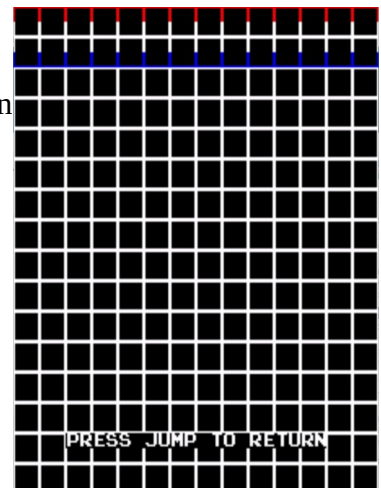
Pattern 0 – Checker Board

This pattern can be used to adjust the drive, contrast, and brightness of the display. If contrast or drive is too high, the colors will “bleed” past the edges of the checker board. If they bleed past it, try turning down the appropriate drive or contrast of the monitor.



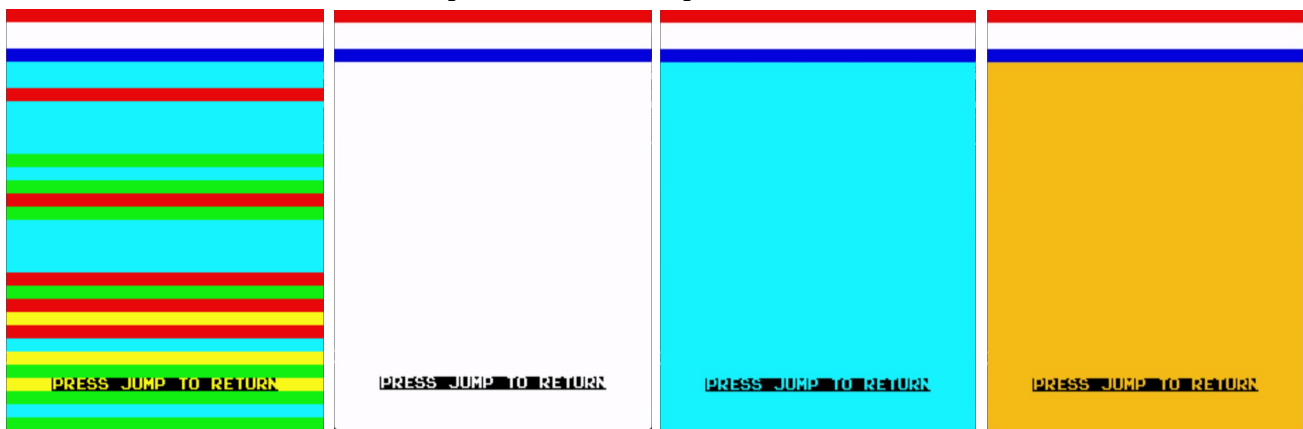
Pattern 1 – Grid

This pattern is useful for adjusting the monitor’s convergence. Convergence issues look like colors separating from each other. They can be very tough to troubleshoot. The Wells Gardner K4900 manual has a good section about convergence adjustments. There is an issue where there are red and blue sections at the top of the screen. This is an issue with TKG hardware due to the fixed color palette for background objects. Unlike the checker board pattern, this could not be easily corrected.



Pattern 2 – Block

This section is to just give a solid background. The background is primarily white so it could be used to adjust the color to not have a hue. It can also be used to verify that there aren’t issues with the color PROMs. The color of the test is dependent on which palette is selected in the runtime menu.



Palette 0

Palette 1

Palette 2

Palette 3

Sound Test

There are seven total dedicated sounds. Some are discretely generated and some are generated by the 8035 MPU. The startup test plays each sound but these can all be selected in the test menu.

Sound ID	Name	Discretely generated or MPU?	Write Address
0	Walking/Shoe Squeak	Discrete	\$7D00
1	Jump	Discrete	\$7D01
2	Crash/Boom	Discrete	\$7D02
3	Spring/Insert Credit	MPU	\$7D03
4	Falling	MPU	\$7D04
5	Point Get	MPU	\$7D05
6	Lose a life	MPU	\$7D80

With the exception of sound 6, sound is selected by a 74LS259 (6H on TKG4). Output 0 is sound 0, output 1 is sound 1, ect. After it goes through a 74LS04 inverter (6J on TKG4).

The discrete sounds then feed into a 74LS05 hex inverter with an open collector output. This is what triggers the discrete sound. The walking and jumping sound have their clock generated by a 556 Dual Timer (8N on TKG4). Sometimes when the 556 fails it can result in either a non-existent sound or a flat sound. The crash sound is has its sporadic clock generated by three 74LS164 shift registers and a 74LS161 binary counter. A failure in the clock circuit can either result in an incorrect sound crash sound or a non-existent one. Each of these sounds also have their own output sections of discrete components. Occasionally these can fail due to a broken or failed electrolytic capacitor. Especially if the PCB is stored incorrectly.

MPU sounds work a bit differently. They are controlled by an 8035 MPU which references two 2716 ROMs. If the ROMs are not good, the MPU will be unable to function. There are also two 74LS74 latch ICs that are involved with the ROM addressing. There is also a little bit of ROM decoding logics. Assuming the MPU is working, it feeds data into a DAC-08. These seldomly fail but it isn't unheard of. The output of the DAC feeds into an MB3614 (LM324 equivalent). The MB3614 is a higher failure component that results in either a quiet or distorted MPU sound set or in some cases no MPU sounds at all. After that it feeds directly into a volume potentiometer and then gets mixed in with the rest of the sounds. The potentiometer sometimes can get damaged if the board is incorrectly stored.

Music Test

Default TKG software supports up to sixteen music tracks. However, two of the music tracks are silent. Pressing the jump button selects the current music track to be played. For the most part, it will repeat until another track is selected. Music is generated by the 8035 MPU on TKG hardware. One thing to keep in mind is that it can also be substituted with an 8039 MPU which is nearly identical other than having more internal RAM. The MPU, when running, reads a 4-bit (\$00-\$0F) written to \$7C00 by the

main CPU and then outputs to an external DAC. The 8035 addresses two external 2716 EPROMs on TKG hardware. If the EPROMs are not functional or are corrupted then the MPU will not work as intended. There is also some addressing logic and latches that are used for addressing the EPROM that are required to be functional. When running the MPU outputs to a DAC-08 (which can be substituted with a modern MC1408). The output of the DAC feeds into an MB3614 (LM324 equivalent). The MB3614 is a higher failure component that results in either a quiet or distorted MPU sound set or in some cases no MPU sounds at all. The output of the opamp goes through a potentiometer for volume adjustment.

Track ID	Sound Description
0	Off
1	Intro
2	Interlude
3	Running low on time
4	Hammer
5	Finished 100m even amount of times
6	Hit enemy with hammer
7	Finished Level
8	25m theme
9	50m theme
A	Off (75m theme)
B	100m theme
C	Finished 100m odd amount of times
D	Jump over enemy
E	100m clear
F	Alert/Gorilla laugh

Background Test

The background objects are things like the text that prints onto the screen. It has a fixed color palette for the whole screen. This test's purpose is to print the entire contents of both video ROMs as it would be used by the game in a 16 x 16 grid. The background object ID can be represented by \$YX where Y is the vertical values and X is the horizontal values of the grid. To return back to the runtime menu press the jump button.

The video ROMs are driven by 2114 SRAMs on the video board in order to make the background object data. It also feeds a MB7052 (82S129) color PROM to account for the color data for the background objects. When the CPU isn't writing to the background objects the RAM



uses the horizontal and vertical counters. In the event that there is issues with the CPU not being able to write to the RAM but the data is not corrupted, then it could be the control logic, 74LS157 Multiplexers, or 74LS245 buffer. If the background objects themselves are corrupted, it is likely an issue with the ROMs themselves or an issue with the data after the ROMs. The ROM data feeds into two 74LS299 shift registers and then gets mixed with the sprite data before going to the CPU board.

Sprite Test

The sprite test simply gives means to print a sprite and give the user the ability to move it around on the screen. The sprite can be changed by pressing P1 and the color palette can be changed by pressing P2. The sprite can be moved around on the screen using the joystick.

The test can be useful for troubleshooting position issues and issues where there are “dead zones” in which the sprite disappears. The sprite data gets written to the 2148 SRAMs on the video board. It’s access circuitry is nearly identical to the video/background RAM. Data from the CPU goes through a 74LS245 buffer and the address lines get mixed in using 74LS157 multiplexers. How the RAM data is used is much more complicated. The data gets latched then aligned with the vertical positions. From there it goes through an 82S09 bipolar RAM. After the bipolar RAM it goes through the ECL RAM setup which is responsible for color and pixel data. It also references the four 2716 ROMs for the actual sprite data. Sprite issues can get very tricky and convoluted quickly. To troubleshoot the data after the RAM refer to the schematics.



Reset

This is not really a test. It’s only purpose is to enter a loop to wait for the watchdog to reset the hardware. If there is no watchdog present, then it will timeout and jump back to \$0000 where all the startup tests will run again. During this waiting period between initiating the reset and the actual reset it will print “reset in progress”.

Chapter 4: Test Architectures

Appendix A: Abbreviations

CPU: Central Processing Unit

DAC: Digital to Analog Converter

DIP: Dual Inline Package

EPROM: Erasable Programable Read Only Memory

IC: Integrated Circuit

MPU: Micro-Processor Unit

N.A.: Not Applicable

NMI: Non-Maskable Interrupt

PCB: Printed Circuit Board

PROM: Programmable Read Only Memory (Also called Bipolar PROM)

RAM: Random Access Memory

ROM: Read Only Memory

Appendix B: ICs mentioned

2114: [1k x 4 SRAM](#)

2148: [1k x 4 High Speed SRAM](#)

74LS04: Hex Inverter

74LS08: [Quad 2-input AND gate](#)

74LS32: [Quad 2-input OR gate](#)

74LS138: [3-line to 8-line decoder/demultiplexer](#)

74LS139: [Dual 2-line to 4-line decoder/demultiplexer](#)

74LS157: [Quad 2-line to 1-line data selector/multiplexer](#)

74LS245: [Octal Bus Tranceiver](#)

8035: 8-Bit MPU in Intel MCS-48 series

8039: 8-Bit MPU in Intel MCS-48 series

8257: DMA Controller

DAC-08: 8-bit digital to analog converter – see MC1408

LM324: Common opamp used before main amplifier in arcade games

MC1408: 8-bit digital to analog converter

Z80: 8-bit CPU